

PRUEBA DE CONCEPTO

Webpack vs Vite

Integrantes

Bitti Guido — 48937

Cosimo Ivo — 52618

Mora Joaquín — 51388

Ulla Lucas — 53246

Profesores

Meca Adrián

Ciancio Damián

25 de agosto de 2026

1. Informe general: Vite vs Webpack

1.1 ¿Qué es un Bundler?

Un module bundler es una herramienta que toma decenas o cientos de archivos independientes de un proyecto (JavaScript, CSS, imágenes, etc.) y los une en unos pocos archivos optimizados para el navegador. Su función principal es resolver los problemas de cargar múltiples archivos por separado: evita cientos de peticiones HTTP lentas, previene conflictos entre variables que comparten el mismo nombre global y se encarga de organizar automáticamente las dependencias en el orden correcto, además de limpiar y comprimir el código para que tu sitio web cargue mucho más rápido.

1.2 ¿Por qué usar Bundlers?

Los bundlers surgieron para solucionar los problemas que aparecen a medida que las aplicaciones web crecen en tamaño y complejidad.

Permiten organizar, procesar y optimizar el código y sus dependencias, mejorando los tiempos de desarrollo y de compilación para producción.

Con el tiempo, estas herramientas han evolucionado para aprovechar nuevas tecnologías y ofrecer procesos de construcción más rápidos y eficientes, convirtiéndose en una parte fundamental del desarrollo de aplicaciones web modernas.

2. ¿Qué es cada tecnología?

2.1 ¿Qué es Vite?

Vite (palabra en francés para "rápido", pronunciado /vi:t/, como "veet") es una herramienta de compilación (build tool) de código abierto para proyectos web modernos. Se utiliza como parte del entorno de desarrollo y construcción de aplicaciones, especialmente en proyectos que utilizan tecnologías como JavaScript, TypeScript y frameworks como React, Vue o Svelte.

Filosofía de Vite

Vite busca ofrecer una herramienta liviana, extensible y enfocada en el rendimiento, apoyada en cuatro pilares:

Núcleo liviano: mantiene una API reducida y un sistema de plugins, evitando incorporar funcionalidades innecesarias al core del proyecto.

Rendimiento: HMR extremadamente rápido, apoyado en herramientas escritas en lenguajes de bajo nivel como Oxc y Rolldown (ver sección de Historia y evolución).

Web moderna: se apoya en ES Modules (ESM) y en los estándares más recientes del navegador, en vez de recrear soluciones que el propio navegador ya resuelve nativamente.

2.2 ¿Qué es Webpack?

Webpack también es un empaquetador de módulos (module bundler) para aplicaciones de JavaScript modernas que funciona procesando todo el proyecto desde un archivo de entrada para construir un grafo de dependencias interno, mediante el cual analiza cada módulo, hoja de estilo o recurso estático necesario para transformarlos y combinarlos en uno o más paquetes (bundles) altamente optimizados para el navegador; gracias a su arquitectura extensible basada en loaders y plugins, ofrece un control granular y extremadamente flexible para transformar sintaxis avanzadas, gestionar assets complejas e implementar estrategias como la división de código (code-splitting).

3. Documentación y comunidad

3.1 Volumen de información

Webpack cuenta con una cantidad de información acumulada significativamente mayor a nivel global debido a sus más de 10 años en el mercado; la red está repleta de tutoriales, hilos en Stack Overflow, artículos en Medium y configuraciones compartidas. Sin embargo, en lo que respecta a sus portales oficiales, la experiencia de lectura de Vite suele resultar mucho más directa y accesible.

3.2 Documentación de Webpack

La documentación de Webpack es enciclopédica, exhaustiva y muy detallada. Explica a fondo cada concepto (loaders, plugins, chunks, manifests), pero esa misma densidad puede abrumar a quienes se inician en la herramienta. A menudo requiere consultar recursos externos de terceros para entender cómo integrar tecnologías modernas que no vienen configuradas por defecto.

- Sitio oficial: <https://webpack.js.org/concepts/>
- Documentación oficial de Module Federation: <https://module-federation.io/>
- SurviveJS (libro y guía avanzada): <https://survivejs.com/webpack/>

3.3 Documentación de Vite

Vite destaca por ofrecer una documentación moderna, ágil y de alta calidad técnica. Al venir con soporte preconfigurado para TypeScript, JSX, CSS y entornos de frameworks como React o Vue, sus guías son cortas y concisas. Cuenta con traducciones oficiales al español, alemán, portugués y japonés directamente desde su sitio principal.

- Sitio oficial (en español): <https://es.vite.dev>
- Sitio oficial (en inglés): <https://vite.dev>

4. Adopción, percepción y evolución de Webpack y Vite

Se analizaron los resultados de la encuesta State of JavaScript 2025, una encuesta realizada a miembros de la comunidad JavaScript que permite observar el nivel de adopción, satisfacción y principales problemas asociados a las herramientas de desarrollo. La sección específica de Build Tools analiza tecnologías utilizadas para la compilación y empaquetado del código. Los gráficos de esta sección fueron generados a partir de los datos oficiales exportados directamente de la encuesta (2025.stateofjs.com/en-US/libraries/build-tools/), aislando únicamente Webpack y Vite para facilitar la lectura comparativa.

4.1 Adopción

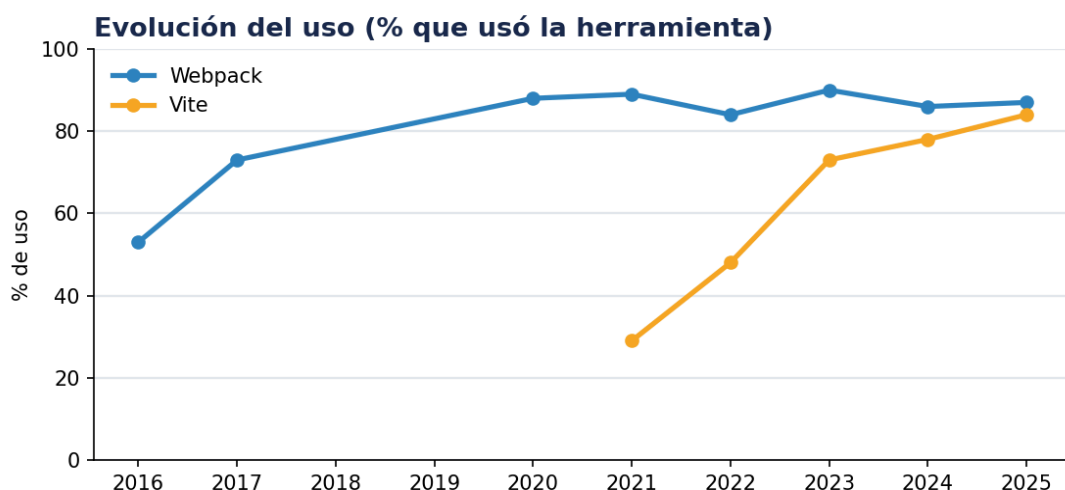
Los resultados de 2025 muestran que Webpack continúa siendo la herramienta con mayor proporción de uso entre los encuestados, con un 87 %, mientras que Vite alcanza un 84 %. La diferencia es de aproximadamente dos puntos porcentuales, por lo que ambas tecnologías presentan actualmente una adopción muy elevada.

Este resultado demuestra que Webpack mantiene una posición sólida dentro del ecosistema JavaScript y que su utilización continúa siendo significativa. Sin embargo, la diferencia reducida con Vite indica que esta última ha alcanzado un nivel de adopción muy cercano al de Webpack.

El propio informe de State of JavaScript señala que se esperaba que Vite superara a Webpack en 2025 como la herramienta de build más utilizada, pero esto finalmente no ocurrió y dicha transición podría producirse en años posteriores.

4.2 Evolución de la adopción

La evolución histórica permite observar mejor la situación de ambas herramientas. Webpack presenta una fuerte presencia acumulada a lo largo de los años, mientras que Vite muestra un crecimiento considerable en los últimos años, prácticamente duplicando su uso desde su primer registro en la encuesta (2021).

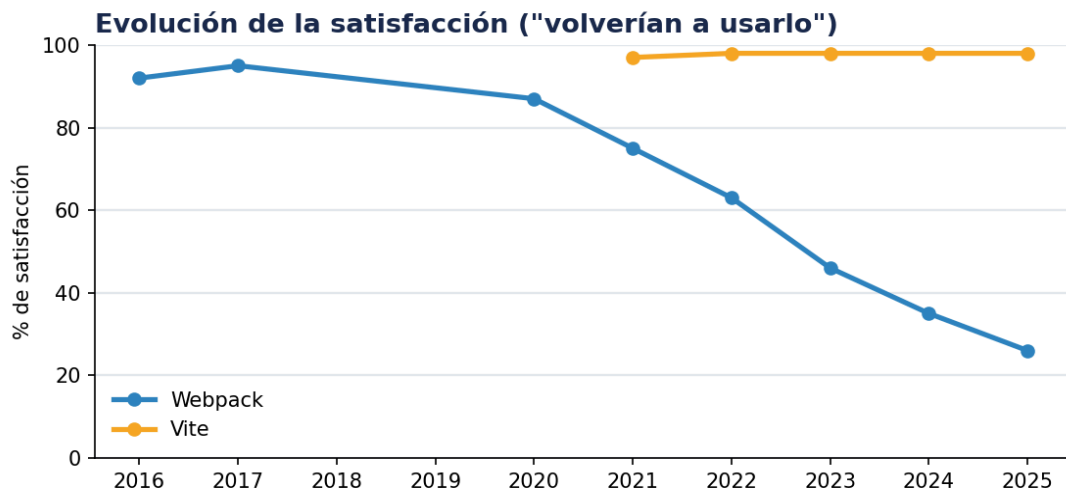


Evolución del porcentaje de encuestados que utilizaron Webpack y Vite entre 2016 y 2025. Fuente: State of JS 2025 (datos exportados).

Este gráfico permite analizar la evolución temporal de ambas tecnologías y observar cómo Vite fue reduciendo progresivamente la diferencia respecto de Webpack. La comparación resulta especialmente relevante porque permite distinguir entre la gran presencia histórica de Webpack y el crecimiento acelerado de Vite.

4.3 Satisfacción de los usuarios

Además del nivel de adopción, State of JavaScript analiza la percepción de los usuarios mediante la proporción de encuestados que, habiendo usado la herramienta, afirman que volverían a usarla. En este aspecto, Vite presenta una valoración marcadamente más favorable que Webpack.

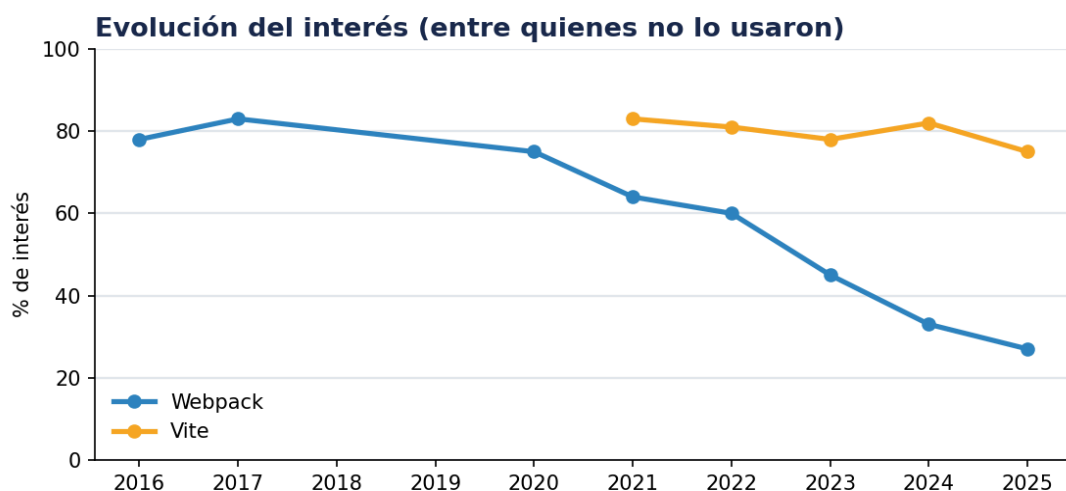


Evolución de la satisfacción ("volverían a usarlo") de Webpack y Vite entre 2016 y 2025. Fuente: State of JS 2025 (datos exportados).

El contraste es notable: mientras Vite se mantuvo estable por encima del 97 % desde su primer registro en 2021, Webpack muestra una caída sostenida año a año, pasando de 92 % en 2016 a apenas 26 % en 2025. Esta diferencia de percepción puede relacionarse con una de las características más importantes de Vite: la reducción de la configuración necesaria para comenzar a desarrollar. Mientras que Webpack ofrece un alto nivel de control y personalización, esto puede implicar una mayor complejidad inicial.

4.4 Interés

Vite mantiene un nivel de interés elevado y estable, ya que la comunidad continúa adoptándolo activamente y sigue de cerca su evolución, no solo utilizando la herramienta, sino también probando betas y reportando bugs antes de que salga una nueva versión.



Evolución del interés (entre quienes no usaron la herramienta) de Webpack y Vite entre 2016 y 2025. Fuente: State of JS 2025 (datos exportados).

Webpack, en cambio, presenta un interés en caída sostenida desde 2017 —de 83 % a apenas 27 % en 2025—, lo que sugiere que cada vez menos desarrolladores manifiestan el deseo de aprenderlo desde cero, a menos que deban mantener proyectos corporativos existentes.

4.5 Conclusión de la sección

A partir de la información analizada, Webpack continúa siendo una tecnología de gran relevancia dentro del ecosistema JavaScript, principalmente debido a su amplia adopción, madurez y capacidad de configuración. El hecho de que en 2025 haya sido utilizado por el 87 % de los encuestados demuestra que continúa teniendo una presencia muy importante.

Por otro lado, Vite presenta una evolución particularmente favorable. Su nivel de utilización ya se encuentra muy próximo al de Webpack, con un 84 %, y el informe destaca su elevada satisfacción entre los desarrolladores y su crecimiento dentro del ecosistema.

Por lo tanto, la principal diferencia no se encuentra simplemente en cuál de las dos tecnologías posee mayor adopción, sino en el enfoque que propone cada una. Webpack se caracteriza por su madurez, flexibilidad y elevado nivel de personalización, mientras que Vite prioriza una experiencia de desarrollo más simple y moderna.

5. ¿Cómo funcionan internamente ambas tecnologías?

A nivel interno, la diferencia fundamental entre Webpack y Vite radica en cómo y cuándo procesan el código fuente para entregárselo al navegador: mientras uno realiza un empaquetado integral previo, el otro transforma los recursos bajo demanda durante el desarrollo.

5.1 Webpack: procesamiento integral y grafo de dependencias

Webpack opera mediante un flujo de compilación centralizado previo a la ejecución. Su proceso comienza en un archivo de entrada principal, desde el cual analiza de manera recursiva cada instrucción de importación para construir un mapa completo de la aplicación denominado grafo de dependencias. Durante este recorrido, utiliza loaders para transformar archivos no estandarizados —como sintaxis de TypeScript, JSX o preprocesadores de CSS— en código JavaScript ejecutable por el navegador. Posteriormente, ejecuta un pipeline de plugins diseñado para aplicar optimizaciones globales, tales como la eliminación de código no utilizado (tree-shaking) y la división estratégica de archivos (code-splitting). Como resultado, genera uno o varios paquetes compilados (bundles) listos para su distribución.

5.2 Vite: módulos nativos bajo demanda

Vite adopta una arquitectura dividida que optimiza por separado la fase de desarrollo y la de producción.

Entorno de desarrollo

El servidor inicia de forma instantánea al prescindir del empaquetamiento previo, aprovechando el soporte nativo de módulos ES (ES Modules) de los navegadores modernos. El navegador solicita los archivos individualmente a medida que los requiere, permitiendo a Vite interceptar cada petición y transformar únicamente el módulo solicitado en tiempo real. Para gestionar eficientemente las dependencias externas almacenadas en `node_modules`, Vite utiliza esbuild (un empaquetador escrito en Go) para pre-construir dichas librerías, dividiendo primero los módulos de una aplicación en dos categorías: dependencias y código fuente.

Entorno de producción (build)

Para la compilación de producción, Vite empaqueta la aplicación en artefactos estáticos altamente optimizados. Es importante aclarar un punto de la arquitectura que evolucionó con el tiempo: originalmente Vite empleaba Rollup para esta etapa; sin embargo, con el desarrollo de VoidZero Inc. (la empresa fundada por Evan You, creador de Vite), a partir de 2026 Vite utiliza Rolldown —un bundler de nueva generación escrito en Rust— como motor de empaquetado para producción, en reemplazo progresivo de Rollup. Esta migración se detalla en la sección de Historia y evolución.

6. Características principales de ambas tecnologías

Las características principales de Webpack y Vite reflejan los objetivos para los cuales fue diseñada cada herramienta, destacando sus diferencias en flexibilidad, rendimiento y facilidad de uso.

6.1 Webpack: control granular y arquitectura extensible

Webpack se distingue por su extraordinaria capacidad de configuración y su adaptabilidad a entornos complejos. Entre sus características principales destaca la gestión integral de todo tipo de recursos (assets) mediante un ecosistema maduro de loaders y plugins, lo que permite procesar imágenes, estilos y fuentes dentro del mismo pipeline de compilación. Ofrece un control exhaustivo sobre la división de código (code-splitting) y la carga diferida (lazy loading), lo que optimiza la entrega de recursos en aplicaciones de gran escala. Asimismo, incorpora capacidades avanzadas como Module

Federation, una arquitectura que permite compartir código y dependencias en tiempo de ejecución entre diferentes aplicaciones independientes (micro-frontends) de manera nativa.

Core Concepts

- **Entry:** Webpack comienza su análisis desde uno o varios puntos de entrada definidos por el desarrollador.
- **Resolución del grafo de dependencias:** a partir del entry, Webpack sigue cada import/require y construye un grafo completo con todos los módulos que el proyecto necesita.
- **Loaders:** por defecto, Webpack solo interpreta JavaScript y JSON. Los loaders son transformadores que permiten procesar otros tipos de archivo (CSS, JSX, TypeScript, imágenes, etc.), convirtiéndolos en módulos válidos dentro del grafo.
- **Plugins:** mientras los loaders transforman archivo por archivo, los plugins actúan sobre el proceso de build completo —por ejemplo, inyectar el bundle en un HTML, extraer CSS a un archivo separado o limpiar la carpeta de salida. Resumidamente, agregan más funcionalidad a Webpack.
- **Output:** finalmente, el grafo resuelto se empaqueta en uno o más archivos de salida, aplicando estrategias como hashing de contenido (para cache busting) y separación en chunks independientes.

Características distintivas adicionales

Loaders y ecosistema: cualquier tipo de archivo se procesa mediante loaders encadenables, con un ecosistema dedicado (webpack-contrib) de cientos de loaders y plugins ya hechos.

Configuración explícita y granular: cada paso del proceso (entry, loaders, plugins, output) se declara a mano, dando control total sobre el build.

Module Federation: permite compartir código entre builds independientes en tiempo de ejecución — clave para micro-frontends.

Asset Modules: manejo nativo de imágenes y fuentes desde Webpack 5, sin loaders externos.

Transpila todo antes de servir: procesa la app completa (código propio + dependencias) antes de poder servir nada, tanto en desarrollo como en producción — a diferencia de Vite, que solo pre-empaqueta las dependencias externas.

6.2 Vite: velocidad de desarrollo y configuración predeterminada

Vite se caracteriza por priorizar la experiencia del desarrollador (Developer Experience) a través de un rendimiento superior y una configuración mínima (zero-config). Su servidor de desarrollo ofrece tiempos de arranque prácticamente instantáneos y un Hot Module Replacement (HMR) de alta velocidad que mantiene su fluidez independientemente del tamaño del proyecto. Incluye soporte nativo e inmediato para TypeScript, JSX, CSS Modules y variables de entorno sin requerir complementos adicionales. Además, proporciona una plantilla de configuración unificada y simplificada basada en un único archivo, combinando la velocidad de desarrollo con la generación de compilaciones de producción altamente optimizadas.

Configuración cero o mínima

Listo para usar: valores predeterminados para los casos más comunes, permitiendo crear un proyecto y comenzar a desarrollar de inmediato, con menos tiempo dedicado a configurar herramientas.

Configuración cuando hace falta: Vite no busca cubrir todos los casos de uso desde el core; las necesidades específicas se resuelven mediante plugins, y el proyecto puede personalizarse a través de un único archivo, vite.config.

Núcleo liviano: mantiene una API reducida y simple, evitando incorporar funcionalidades innecesarias al núcleo — esto favorece un proyecto sostenible y extensible en el tiempo.

Hot Module Replacement nativo

El sistema de HMR de Vite detecta qué módulo fue modificado y actualiza únicamente ese módulo, sin recargar toda la página y manteniendo el estado de la aplicación. Su API de HMR está basada en ESM nativo, con integración oficial para los frameworks más usados:

- Integración oficial con React, Vue y Preact.
- React Fast Refresh y Vue HMR incluidos como soporte de primera clase.
- Viene preconfigurado automáticamente al iniciar un proyecto con create-vite, sin necesidad de instalar ni configurar plugins adicionales de forma manual (a diferencia de Webpack, donde el equivalente —React Refresh— debe agregarse explícitamente, tal como se detalla en la sección de configuración real de Webpack).

7. Historia y evolución

7.1 Webpack

Webpack nació como un proyecto personal de Tobias Koppers en 2012, a partir de su tesis de maestría sobre la optimización de aplicaciones web. Su objetivo original era implementar code splitting, es decir, dividir una aplicación en fragmentos que pudieran cargarse bajo demanda. Como no logró incorporar esta idea a Modules Webmake, una herramienta que intentaba extender, Koppers decidió crear su propia solución. El release inicial de Webpack se publicó el 19 de febrero de 2014.

Su principal factor diferencial no fue la velocidad, sino su enfoque de tratar todo el frontend como un módulo: no solo JavaScript, sino también CSS, SVG y fuentes, mediante un sistema de loaders. En 2019, Webpack pasó a formar parte de la OpenJS Foundation, lo que permitió institucionalizar su gobernanza. Más adelante, en 2022, Koppers, ya trabajando en Vercel, presentó Turbopack, un bundler escrito en Rust y concebido como sucesor de Webpack, con Next.js como su primer cliente.

Para 2026, Webpack publicó un roadmap hacia la versión 6, liderado por Even Stensberg, que contempla características como soporte nativo para CSS Modules, un "universal target" capaz de compilar para distintos entornos y soporte integrado de TypeScript.

7.2 Vite

Vite fue creado por Evan You, también autor del framework Vue.js, con su primer lanzamiento el 20 de abril de 2020. En sus inicios estaba fuertemente ligado al ecosistema de Vue, pero con el lanzamiento de Vite 2.0 en 2021 se convirtió en una herramienta agnóstica de framework, con un núcleo independiente y soporte para distintos frameworks a través de plugins (Vue, React, Preact, Lit, entre otros). A partir de ahí su adopción creció de forma acelerada: para 2024 ya superaba los 15 millones de descargas semanales, consolidándose como la herramienta de referencia para frameworks como Vue, Svelte y React.

En 2024, You anunció la creación de VoidZero Inc., una compañía dedicada a construir un toolchain unificado para el ecosistema JavaScript, apoyándose en la base de usuarios que ya tenía Vite. Bajo este proyecto se desarrollan herramientas complementarias escritas en Rust, como Rolldown (bundler) y Oxc (parser, transformador y linter), pensadas para reemplazar progresivamente a Rollup y esbuild dentro de Vite. Esta evolución ya se refleja en las versiones más recientes: hacia 2026 Vite utiliza Rolldown internamente para el empaquetado de producción, marcando un cambio respecto a su arquitectura original basada en Rollup y esbuild (ver sección 5.2). El proyecto también generó su propio evento de comunidad, ViteConf, celebrado anualmente desde 2022.

8. Gobernanza y estado actual

8.1 Webpack

Webpack cuenta en su sitio oficial con una sección de patrocinadores ("Support the Team"), organizada en niveles de contribución: Latest Sponsors, Platinum, Gold, Silver, Bronze y Backers. Según el propio sitio, estos aportes —vía contribuciones, donaciones y patrocinio— financian directamente las horas de trabajo del equipo, mejoras continuas del proyecto y, especialmente, la documentación y el material de aprendizaje.

El proyecto opera bajo el paraguas de la OpenJS Foundation, que además de alojar a Webpack sostiene a otros proyectos clave del ecosistema JavaScript (como Node.js y Electron), lo que le da a Webpack un respaldo institucional formal más allá de su comunidad de código abierto. La toma de decisiones se estructura en dos niveles: un Technical Steering Committee (TSC), que define la

dirección técnica y estratégica del proyecto, y un Core Working Group, que sostiene el desarrollo y mantenimiento del día a día.

Patrocinadores corporativos (Platinum/Gold/Silver): empresas que inyectan capital mensual significativo, obteniendo visibilidad en la documentación oficial y repositorios. Ejemplos: Google, Trivago, Airbnb, Shopify, Microsoft, Discord, Notion, Google Angular y JavaScript Fund.

Respaldos individuales (Bronze/Backers): pequeñas empresas o desarrolladores de la comunidad que realizan aportes menores.

8.2 Vite

A diferencia de Webpack, que está institucionalizado bajo la OpenJS Foundation, Vite no depende de una fundación sin fines de lucro sino de una empresa privada con capital de riesgo: VoidZero Inc. VoidZero recibió un financiamiento inicial de 4,6 millones de dólares liderado por Accel, con participación de Amplify Partners, lo que generó cierto debate en la comunidad open source sobre el modelo de negocio detrás de una herramienta gratuita tan popular.

El plan de monetización de VoidZero no pasa por cobrar por Vite en sí, sino por ofrecer soluciones end-to-end orientadas a empresas, construidas sobre los componentes open source ya liberados; de hecho, en 2025 se anunció Vite+, una herramienta paga pensada para entornos corporativos.

Pese a este respaldo comercial, el desarrollo del proyecto sigue siendo abierto: Vite 6 fue posible gracias a la contribución de más de 1000 colaboradores, y hoy es utilizado por compañías como OpenAI, Google, Microsoft, Apple y Shopify.

Este modelo de empresa privada combinada con código abierto es un buen contraste para la conclusión general: mientras Webpack cuenta con un respaldo institucional formal (fundación sin fines de lucro más sponsors corporativos tradicionales), Vite avanza con un modelo más moderno de "empresa de producto open source" financiada por capital de riesgo, típico de las herramientas nacidas en los últimos años del ecosistema JavaScript.

9. Características de evaluación e investigación comparativa

9.1 Características de evaluación

Para comparar Webpack y Vite de forma objetiva, se definió un conjunto de características agrupadas en dos categorías: técnicas (medibles de forma directa sobre la implementación) y de proceso (relativas al ecosistema y la experiencia de desarrollo en general). La siguiente tabla lista las características seleccionadas y el método utilizado para evaluarlas.

Categoría	Característica	Cómo se mide
Técnica	Tiempo de build de producción	Cronometrado (2 corridas)
Técnica	Tiempo de arranque del dev server	Cronometrado (2 corridas)
Técnica	Tamaño del bundle final (sin source maps)	du sobre /dist
Técnica	Code splitting por ruta	Inspección de /dist
Técnica	Hot Module Replacement	Prueba manual en vivo
Proceso	Complejidad de configuración (líneas y archivos)	Conteo directo (wc -l)
Proceso	Cantidad de dependencias de build	Conteo en package.json
Proceso	Documentación disponible	Revisión cualitativa
Proceso	Comunidad y ecosistema de plugins	Encuestas públicas (State of JS)
Proceso	Oferta y demanda de desarrolladores	Revisión cualitativa
Proceso	Soporte y mantenimiento a largo plazo	Revisión de gobernanza/roadmap

Para las características técnicas se utilizaron los dos proyectos equivalentes (CRUD de tareas en React, con las mismas rutas, mismo consumo de API vía json-server y misma estructura de componentes) implementados durante la etapa 4 del enunciado de la PoC. Para las características de proceso se combinó revisión cualitativa propia con datos de encuestas públicas de adopción.

Repositorio de las aplicaciones utilizadas: <https://github.com/IvoCosimo/PoC-BuildTools-Bundlers>

9.2 Resultados técnicos medidos

Esta sección presenta los resultados obtenidos al investigar y ejecutar ambas tecnologías sobre los proyectos equivalentes. Las mediciones técnicas se realizaron sobre los mismos dos proyectos, variando únicamente la herramienta de build. Todas las corridas se ejecutaron en la misma máquina (Windows), una inmediatamente después de la otra, para minimizar diferencias de entorno entre ambas tecnologías.

Métrica	Webpack	Vite
Build de producción (corrida 1 / corrida 2)	15.90 s / 2.75 s	1.11 s / 0.83 s
Arranque del dev server (compilación)	2.36 s / 1.55 s	314 ms / 294 ms / 277 ms
Bundle final sin source maps	215 KB	213 KB
Archivos de configuración	4 archivos	1 archivo
Líneas de configuración	≈ 100 líneas	≈ 15 líneas
Dependencias de build (devDependencies)	15	4
Code splitting por ruta	Sí (splitChunks + React.lazy)	Sí (nativo, import() dinámico)

9.3 Análisis de los resultados

Tiempo de build: se observó una variabilidad importante entre corridas en Webpack (15.90 s en la primera corrida, 2.75 s en la segunda), probablemente asociada a costos de arranque en frío del proceso (lectura inicial de `node_modules`, antivirus, cachés del sistema operativo) más que al propio bundler. Tomando la corrida en caliente como referencia más representativa, Vite igualmente resultó considerablemente más rápido (0.83–1.11 s vs 2.75 s, es decir aproximadamente 3 veces más rápido). La diferencia se explica por la arquitectura: Vite usa esbuild (escrito en Go) para pre-empaquetar dependencias y, actualmente, Rolldown para el build final, mientras que Webpack recorre y transforma el grafo completo con su propio motor en JavaScript.

Arranque del dev server: Webpack tardó entre 1.55 s y 2.36 s en compilar antes de quedar listo, mientras que Vite estuvo consistentemente por debajo de los 320 ms en sus corridas (277–314 ms), casi un orden de magnitud más rápido. A diferencia del build de producción, acá la diferencia fue estable y reproducible en todas las corridas, lo que la vuelve la métrica más contundente de la comparación. Esto es consistente con la diferencia arquitectónica documentada en la sección teórica: Webpack genera un bundle completo antes de servir, mientras que Vite sirve los módulos de la aplicación como ES modules nativos y solo pre-empaqueta las dependencias de `node_modules` una vez.

Tamaño del bundle: contrario a una creencia extendida, en este proyecto el tamaño final del bundle (sin source maps) fue prácticamente idéntico entre ambas tecnologías (215 KB en Webpack vs 213 KB en Vite). Esto sugiere que, para proyectos de tamaño chico a mediano como el de esta PoC, el tamaño de bundle no es un diferenciador relevante entre ambas herramientas; la diferencia se concentra en velocidad de build y experiencia de desarrollo, no en el peso final entregado al usuario.

Complejidad de configuración: Webpack requirió 4 archivos de configuración (config común, desarrollo, producción y Babel) con un total aproximado de 100 líneas, además de 15 dependencias de build. Vite logró el mismo resultado funcional con un único archivo de 15 líneas y solo 4 dependencias de build. Esta diferencia es consistente con lo relevado en la sección teórica, donde la complejidad de configuración aparece como el punto de dolor más citado por los usuarios de Webpack en encuestas de la comunidad.

Paridad funcional: ambos proyectos lograron replicar exactamente las mismas características: code splitting por ruta (mediante chunks separados), CRUD completo contra una API simulada con json-server, y proxy del dev server hacia esa API para evitar problemas de CORS. Esto confirma que la comparación es representativa, ya que ambas implementaciones son funcionalmente equivalentes.

9.4 Conclusión de la investigación

Los resultados medidos sobre proyectos equivalentes confirman lo señalado por la comunidad en encuestas públicas: la principal ventaja de Vite sobre Webpack no está en el tamaño del artefacto final —que resultó equivalente en este caso— sino en la velocidad de desarrollo (arranque y rebuild) y en la simplicidad de configuración. Webpack, por su parte, sigue siendo una alternativa sólida cuando el proyecto requiere flexibilidad de configuración avanzada o ya cuenta con una base de código y ecosistema construido alrededor suyo.

10. Entornos para la implementación de cada tecnología

A nivel de arquitectura de software, la decisión debe alinearse con la naturaleza y el ciclo de vida de la organización:

Cuándo elegir Webpack: es la elección lógica para proyectos enterprise existentes o para arquitecturas sumamente complejas que requieren transformaciones de bajo nivel altamente customizadas a través de su inmenso ecosistema de loaders y plugins. Su nivel de madurez y soporte corporativo garantizan que siga siendo una pieza fundamental del ecosistema frontend por los próximos años.

Cuándo elegir Vite: es el ganador indiscutido para cualquier proyecto nuevo (greenfield). Su enfoque radical en la experiencia del desarrollador, su configuración zero-config para casos comunes y la inmediatez de su Hot Module Replacement reducen la fricción inicial casi a cero, traducándose en menores tiempos de salida al mercado y una mayor productividad de los equipos de ingeniería.

11. Conclusión general

Esta Proof of Concept permitió comparar Webpack y Vite tanto desde la investigación documental como desde la implementación práctica de dos proyectos equivalentes. Los resultados obtenidos en ambos frentes —el análisis de encuestas públicas de la comunidad y las mediciones propias sobre proyectos idénticos— apuntan consistentemente en la misma dirección.

Webpack se sostiene como una tecnología madura, extensible y con un ecosistema sin comparación, respaldada institucionalmente por la OpenJS Foundation. Es la opción más sólida para proyectos que ya existen, que requieren configuraciones avanzadas o arquitecturas como Module Federation. Su principal debilidad, tanto en las mediciones propias como en la percepción de la comunidad, es el costo de esa flexibilidad: mayor complejidad de configuración y tiempos de desarrollo más lentos, especialmente notorios en el arranque del dev server.

Vite, por su parte, prioriza la experiencia de desarrollo por sobre la personalización exhaustiva. Su arquitectura basada en ES Modules nativos durante el desarrollo, combinada con esbuild y Rolldown para las tareas de empaquetado, le permite ofrecer tiempos de arranque casi instantáneos sin sacrificar el tamaño del artefacto final de producción, que resultó equivalente al de Webpack en esta PoC. Este enfoque, sumado a un modelo de configuración mínima, explica su creciente adopción y su marcada ventaja en satisfacción de los desarrolladores.

En definitiva, no se trata de determinar cuál tecnología es objetivamente superior, sino de identificar cuál resulta más adecuada según el contexto del proyecto: la madurez y el control granular de Webpack frente a la velocidad y simplicidad de Vite.

Referencias

- [WEBPACK 5 - ¡Curso práctico DESDE CERO! 📦](#)
- [State of JavaScript 2025: Build Tools](#)
- <https://webpack.js.org/>
- [Vite vs Webpack: Una comparativa detallada | by OscarBravelop | Medium](#)
- [Webpack: ¿Qué es y cual es su funcionamiento? | Blog de Arsys](#)
- [Introducción | Vite](#)
- [Filosofía del Proyecto | Vite](#)
- [Por qué usar Vite](#)
- [GitHub - vitejs/vite: Next generation frontend tooling. It's fast!](#)
- Repositorio de las aplicaciones desarrolladas para esta PoC:
<https://github.com/IvoCosimo/PoC-BuildTools-Bundlers>